A layered Bloom filter[23] (LBF) also uses multiple Bloom filters, but organized in layers. Each layer is updated for a key only after the previous layer has already stored the same key. Layered Bloom filters are normally used to implement a counting filter: an LBF with `R` levels can count up to `R` insertions of the same key. Often deletion is also supported.

Each call to `contains` starts by checking the layers, from the closest to the deepest, and returns the index of the last layer where the key was found, or `-1` (or, equivalently, `false`) if the key is not stored in the first layer.

For what concerns storing a key, the `insert` method stores it in the first layer for which contain returns `false`.

Assuming that each layer has a false-positive ratio equal to P_F, and that all layers use different sets of hash functions, then if an element has been stored `c` times on the filter:

- The probability that `contains` returns `c+1`, a counter 1 unit larger that the real number of times a key was stored, is approximately P_F;
- The probability of returning `c+2` is P_F^2;
- Similarly, for `c+d`, the probability becomes P_F^d.

Those, however, are approximate (optimistic) estimates, because the assumptions about the universality and independence of the hash functions are hard to guarantee; to compute the exact probabilities, we would need to take into account the depth and the number of layers.